🛠️ 🤖

WALL-E AI Platform

Super Master Guide — Kel Shi bMalaf Wahad

Blueprint + Corrections + BOM + Wiring + Firmware + Troubleshooting + STL — Muhad Kamel w Musahhah

Consolidated min: Master Blueprint · Corrected Guide (x3) · Structural Sketch · K-system Manual · Chat additions (PCA9685, Boost Converter, INA219, STL blanks)

Fihres (Table of Contents)

1. Muqaddima — Wein Tebda2?

Heda el-malaf howe el-nuskha el-nihaiye elli bteje3 kel el-ma3loumat min kel el-malafat (Master Blueprint, Corrected Guide, Structural Sketch, K-system manual) + kel el-tashih w el-ezafat elli sar ne7kihon bel-chat (PCA9685, Boost Converter, INA219, STL blanks). Iza fi ay ta3arod been malaf w tene, hon mawjoud el-nuskha el-sa77, mesh el-2adime.

Wein tebda2 be-3ajale (suggested first move)

1. **Yom 1 — bala masare:** 2ra2 seksyon 2 (el-qararat) w 7edded: battery wa7de wala tinten, switch, DRV8833 wala TB6612FNG. Hayde qararat lezim ta5odon 2abel ma teṭlob ay 2ata3.
2. **Yom 1-2:** eṭlob el-BOM el-kamel (seksyon 3) min ECA.ir / Roboeq.ir — 3-5 ayyam she7en.
3. **Bel-wa2et elli el-2ata3 3am toṣal:** jahhez el-STL files (seksyon 9) w eb2a jahez lal-ṭbaa3a.
4. **Lamma toṣal el-2ata3:** ebda2 Phase 1 (ṭbaa3a) → Phase 2 (mecanique) → Phase 3 (kahraba) → Phase 4 (barmaje) — be-hal tarteeb, mesh gheiro.

Muhem: la-trekkeb kel el-kahraba 2abel ma teste5dem el-breadboard test (Phase 3, khatwe 11 taht) — kel 7ele lahalha 2abel el-tarkib el-nihaii jowa el-chassis. Heda bi wafferlak wa2et w laḥam iza fi ghalta.

2. Qararat Lazim Tak-hodon Qabel Ma Tebda2 (Phase 0)

Hal qararat bta2thir 3al BOM, el-taklife, w el-wazn — lezim tetakhad 2abel ma teṣref masare.

Qarar	Khiyarat	Ta2thir	Tawsiye
Battery	1× 18650 (3500mAh) aw 2× 18650 parallel (7000mAh)	Runtime: ~3.2 sa3a vs ~6.4 sa3a. Wazn w 3ard el-qa3ide bikaffo la-tinten.	**2× 18650 parallel** — hamesh amen a3la lal-current, runtime aṭwal, farq se3er bsit (~$5 ezafi)
Motor Driver	DRV8833 (asli) aw TB6612FNG (badil)	TB6612FNG: ka2a2a a3la, ḥarara a2al, muqawame dakhiliyye a2al = mawtoor a2wa shwaye	DRV8833 mnih w rakhis — TB6612FNG iza el-mizaniyye btesma7 (+$1-2)
Power Switch	bala switch (lazim tefṣol el-battery yadawiyyan kel marra) aw SPDT slide switch	bala switch = mukhaṭara qaṣar da2ira w mesh 3amali	**Ḍif SPDT switch** — $0.5 bas, w bi7mik el-battery
PTC Fuse	bala fuse aw PTC resettable fuse (500mA-1A)	Ḥimaye basiṭe men qaṣar da2ira athna2 el-lahem	Ḍif — $0.3 bas, ra7iṣa w mufide

3. El-BOM el-Nihaii el-Kamel + el-Taklife

Heda el-BOM el-muwaḥḥad — kel el-ta3dilat (+PCA9685, +Boost Converter, +Capacitor, +Switch, +Fuse, −SD Module el-khareji) mudmaje.

2ata3	Specs	Taklife ($)	Mulahaza
XIAO ESP32S3 Sense	Mic + Camera + PSRAM 8MB	~$12	El-3a2el (brain) — PSRAM lezim yenfa3el bel-Arduino IDE
TFT Display	1.5" SPI ST7789 240×240	~$5	Ta2akkad VCC range bel-datasheet (3.3-5V)
Servos	2× SG53/SG90	~$4	Pan + Tilt
Track Drive	2× N20 Gear Motors + DRV8833	~$6	aw TB6612FNG badil (+$1-2)
Arms 🆕	2× SG90 micro-servo + brass wire/PLA arm structure	~$5	**Ḍaruri** — kanou madhkourin bel-image prompt bas bala spec handasi. Shouf Seksyon 4.1
Audio	MAX98357A I2S Amp + 3W Speaker	~$4	Ta2akkad impedance (4Ω/8Ω) 2abel el-shira2
Distance Sensor	VL53L0X ToF	~$3.5	I2C — nafs bus ma3 PCA9685 (3unwan mukhtalef)
Touch Button	TTP223	~$1	⚠ ghazziya men 3.3V bas — mesh men 5V pin
Power System	1-2× 18650 + TP4056	~$5-10	Ḥasab 3adad el-batariyyet elli 2arrarto
Wireless Charger	5V Rx + Tx coil kit	~$3.5	Ekhtiyari
Expansion Sensors	AHT20 (temp/humidity) + MPU6050 (gyro)	~$3.5	Ekhtiyari — I2C
PCA9685 🆕	16-ch I2C PWM driver	~$2	**Ḍaruri** — bye7el mushkilet el-pin budget w el-motor direction
Boost Converter 🆕	MT3608 (2A rated) aw TP4056+Boost combo	~$1	**Ḍaruri** — bysahhe7 khatt el-5V (battery el-kham bteṭla3 3.0-4.2V mesh 5V thabet)
Capacitor 🆕	100-470µF electrolytic	~$0.3	Byemna3 brownout reset lal-ESP32 wa2et 7arakat el-mawtoor
Power Switch 🆕	SPDT slide	~$0.5	Tashghil/eṭfa2 bala fakk aslak
PTC Fuse 🆕	500mA-1A resettable	~$0.3	Ḥimaye qaṣar da2ira
~~SD Module khareji~~	~~MicroSD SPI Module~~	~~$2.5~~	✖ El-Sense fiyo microSD slot mabniyye — mesh mehtaje wa7de khariji

TOTAL (battery wehde): ~$55-57 **TOTAL (2 batariyyet): ~$60-62**

Se3ar be-Toman: ḍrob be-mu3addal el-ṣarf el-yawmi (taqriban 68,000-70,000 Toman lal-Dollar bḥasab el-suq el-ḥor — te2akkad men se3r el-yom 2abel el-ṭalab).

> **Wein teshtri (Mashhad/Iran):** ECA.ir w Roboeq.ir bighaṭṭo tقريبا kel el-2ata3, be-she7en Tipax/Post la-Mashhad, 3-5 ayyam.

REVISION 2 — 4. Pinout + 4.1 Arms Engineering

Purpose: This revision replaces the ambiguous arm geometry and cleans the channel/power mapping so the design can be translated into CAD and firmware without guessing.

4. Pinout — Final Working Map

PCA9685 CH	Function	Connected hardware	Power / note
CH0	Head Pan	SG53/SG90	Servo V+ = 5V
CH1	Head Tilt	SG53/SG90	Servo V+ = 5V
CH2	Left motor input A	DRV8833 AIN1	PWM logic
CH3	Left motor input B	DRV8833 AIN2	PWM logic
CH4	Right motor input A	DRV8833 BIN1	PWM logic
CH5	Right motor input B	DRV8833 BIN2	PWM logic
CH6	LEFT ARM	SG90	NEW — 5V servo rail
CH7	RIGHT ARM	SG90	NEW — 5V servo rail
CH8–CH15	Reserved	—	Keep available for expansion

4.1 — Arms: complete engineering specification

Item	Final specification
Servo	2 × SG90 micro-servo, nominal 5V; one per arm
Axis	Single-axis rotation at the body-side servo; lightweight link/hand
Arm link	PLA printed link or 1–1.5 mm brass wire reinforcement; target moving length 30–35 mm from servo horn center
Bracket	Side-mount PLA bracket; 3.0 mm nominal wall/plate thickness
Bracket envelope	24 mm wide × 20 mm high × 9 mm deep nominal; verify against the actual SG90 body before printing final STL
Mount holes	2 × Ø2.2 mm pilot/clearance holes for M2 screws; exact SG90 hole spacing must be checked against the physical servo
Body mounting height	Servo center approximately 7 mm from body side and ~24–25 mm above the body base; final position must preserve track/scre
Safe motion	Start with 45°–135°; HOME = 90°. Do not command 0°/180° until physical clearance is verified.
Clearance	Minimum target ≥5 mm from tracks, screen bezel, and body edges through the full sweep
Weight target	Keep moving arm assembly light; target ≤15–20 g per arm including printed parts
Electrical	PCA9685 CH6/CH7; servo V+ from regulated 5V rail; common GND

Important CAD rule: the dimensions above define the intended design envelope, not a replacement for measuring the physical SG90. Make the bracket parametric and leave ±0.5 mm adjustment where practical.

REVISION 2 — 4.1A Arm CAD + Sensor Bus Decisions

Arm bracket / link manufacturing sketch

Part	Nominal dimensions / features	Manufacturing note
Side bracket	24 W × 20 H × 9 D mm; 3 mm plate	2 × Ø2.2 mm holes; fit to actual SG90
Arm link	32 L × 8 W × 3 T mm; Ø2.2 mm end holes	PLA; keep light
Horn interface	SG90 horn + M2/M2.5 fastener as available	Do not force a screw into the horn
Motion	45° DOWN / 90° HOME / 135° UP	Calibrate after horn installation
Clearance	≥5 mm at all positions	Check with tracks moving by hand

A4 printable reference — what must be drawn

Create a true 1:1 A4 sketch for the side bracket and arm link. Mark the two mounting holes, centerline, pivot/horn axis, outer envelope, and 5 mm keep-out zones. The previous PDF only described the arm concept; this page makes the missing manufacturing intent explicit.

I²C bus — fixed addresses for the base build

Device	Address	Status	Rule
PCA9685	0x40	Required	Keep as-is
INA219	0x41	Required for battery/current measurement	Move/configure away from 0x40
VL53L0X ToF	0x29	Required	Single ToF sensor
TCS34725	0x29	Optional	Same address as ToF — only use with XSHUT/mux strategy
AHT20	0x38	Optional	No conflict
MPU6050	0x68	Optional	No conflict
BH1750	0x23	Optional	No conflict

Decision: the base build does not require TCS34725. If it is added later, implement an explicit XSHUT/I²C-multiplexer plan rather than assuming both 0x29 devices can coexist directly.

INA219 placement

For battery percentage/current to mean battery data, place the INA219 current path on the raw battery supply (or on a clearly defined battery-side rail), not on the regulated 5V output. The firmware must label the measured quantity accordingly.

REVISION 2 — 5. Power Architecture (Corrected)

Final power tree

```
18650 battery pack
      |
      +----> TP4056 charger/protection
      |
      +----> INA219 (battery-side measurement)
      |
      +----> DRV8833 VM ----> N20 left/right motors
      |
      +----> MT3608 Boost ----> regulated 5V BUS
                                  |
                                  +--> XIAO ESP32S3 Sense 5V
                                  +--> PCA9685 V+ -> Pan/Tilt + Arms
                                  +--> ST7789 (per module voltage spec)
                                  +--> MAX98357A (per module voltage spec)

All grounds: XIAO + PCA9685 + DRV8833 + sensors + audio + Boost + battery system
must share a common GND reference.
```

Power rules

Rule	Implementation
5V regulation	Set MT3608 to 5.0V with a multimeter before connecting sensitive loads.
Servo rail	Pan/Tilt + CH6/CH7 arms use the regulated 5V rail through PCA9685 V+; do not power servos from XIAO 3.3V.
Motor rail	DRV8833 VM can use the raw battery rail within the driver's allowed range; do not feed motor current through the XIAO/PCA968
Bulk capacitor	100–470 µF across 5V/GND close to servo load; add local decoupling per module datasheets.
Fuse	Keep the planned 500 mA–1 A PTC only if the selected system's real current budget supports it; motors/servos can create startu
Common ground	Required for stable PWM/I²C/serial references.
Bring-up	Test 5V rail unloaded, then with one servo, then two servos, then audio/display, before full robot load.

Correction versus the older wording: TP4056 alone does not create a stable 5V output. The Boost converter is the element that creates the regulated 5V rail.

REVISION 2 — Phase 2/3/4 Bring-up Sequence

Phase 2 — Mechanical assembly

1. Center each SG90 at 90° before attaching its horn.
2. Install the side bracket and verify the servo shaft is square to the body.
3. Attach the lightweight 30–35 mm arm link.
4. Manually sweep the arm through 45°–135° with power OFF.
5. Confirm ≥5 mm clearance from tracks, screen, and body.
6. Only after clearance passes, tighten the horn/link fastener.

Phase 3 — Electrical bring-up

1. Verify the 5V rail with a multimeter. 2. Connect common GND. 3. Connect PCA9685 I²C. 4. Connect CH0–CH7 loads one group at a time. 5. Add the bulk capacitor near the servo rail. 6. Power the DRV8833 motor rail separately from the logic/servo current path. 7. Test each arm individually before running both together.

Phase 4 — Firmware bring-up

Use a single master firmware project. Test in this order: I²C scan → PCA9685 → one servo → two arm servos → head servos → motors → ToF → display → INA219 → audio → Wi-Fi/dashboard. Do not enable AI voice until the local robot control layer is stable.

Firmware architecture

```
setup()
  ├─ initPowerMonitor()
  ├─ initPCA9685()
  ├─ initServos()
  ├─ initMotors()
  ├─ initToF()
  ├─ initDisplay()
  ├─ initAudio()
  └─ initNetwork()

loop()
  ├─ updateSensors()
  ├─ processCommands()
  ├─ safetyCheck()
  ├─ updateRobotState()
  └─ serviceWebServer()
```

Robot state machine

Use explicit states such as IDLE, MOVING, WAVING, ARMS_UP, AVOIDING, and ERROR. The AI or dashboard requests an intent; the firmware validates it and then calls the corresponding hardware function.

REVISION 2 — 7. Troubleshooting + Safety

Problem	Likely cause	Corrective action
Arm does not move	Wrong CH6/CH7, no 5V, no common GND	Check PCA9685 channel, 5V V+, GND, and servo signal
Arm jitters	Voltage droop/noise	Verify 5V rail; add 100–470 µF bulk capacitor close to servo rail
Arm stalls / heats	Mechanical binding or excessive angle/load	Power off; check clearance; reduce to 45°–135°; lighten arm
Arm hits body/track	Bracket position or limits wrong	Reposition bracket; verify ≥5 mm clearance across full sweep
ESP32 brownout	Servo/motor peak current	Separate motor current path; verify Boost output; add bulk capacitance
Motors weak	Motor rail voltage/current issue	Measure DRV8833 VM under load; do not feed motor current through logic rail
I^2C device missing	Address conflict/wiring	Run I^2C scanner; PCA9685 0x40, INA219 0x41, ToF 0x29
Battery % wrong	INA219 measuring boosted 5V or voltage-only approximation	Move INA219 to battery-side measurement path and calibrate mapping
Dashboard blocks motion	Long delay() inside handlers	Keep animations non-blocking in final firmware or move them to state machine
AI command unsafe	AI directly drives hardware	AI sends high-level intent only; firmware applies limits/safety checks

Hard safety limits

Never let the AI, web client, or external command write arbitrary servo angles or raw PWM. All incoming commands pass through clamped functions such as setArmAngleSafe(), driveMotorSafe(), and emergencyStop().

REVISION 2 — 8. Firmware Core (Single Master .ino)

This is the corrected core structure. It removes the multiple-setup problem in the previous guide and gives the arms safe limits.

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <Adafruit_VL53L0X.h>
#include <Adafruit_INA219.h>
#include <TFT_eSPI.h>

Adafruit_PWMServoDriver pwm(0x40);
Adafruit_VL53L0X lox;
Adafruit_INA219 ina219(0x41);
TFT_eSPI tft;

#define PAN_CH      0
#define TILT_CH     1
#define L_AIN1      2
#define L_AIN2      3
#define R_BIN1      4
#define R_BIN2      5
#define LEFT_ARM_CH 6
#define RIGHT_ARM_CH 7

#define SERVO_MIN_PULSE 150
#define SERVO_MAX_PULSE 600

#define ARM_DOWN 45
#define ARM_HOME 90
#define ARM_UP 135

enum RobotState {
  IDLE, MOVING, WAVING, ARMS_UP, AVOIDING, ERROR_STATE
};

RobotState robotState = IDLE;

void setServoAngle(uint8_t ch, int angle) {
  angle = constrain(angle, 0, 180);
  int pulse = map(angle, 0, 180,
                  SERVO_MIN_PULSE, SERVO_MAX_PULSE);
  pwm.setPWM(ch, 0, pulse);
}

void setArmAngleSafe(uint8_t ch, int angle) {
  angle = constrain(angle, ARM_DOWN, ARM_UP);
  setServoAngle(ch, angle);
}

void armsHome() {
  setArmAngleSafe(LEFT_ARM_CH, ARM_HOME);
  setArmAngleSafe(RIGHT_ARM_CH, ARM_HOME);
}

void emergencyStop() {
  // Stop both motor input pairs
  pwm.setPWM(L_AIN1, 0, 0);
  pwm.setPWM(L_AIN2, 0, 0);
  pwm.setPWM(R_BIN1, 0, 0);
  pwm.setPWM(R_BIN2, 0, 0);

  armsHome();
  setServoAngle(PAN_CH, 90);
  setServoAngle(TILT_CH, 90);
  robotState = IDLE;
}

void setup() {
  Serial.begin(115200);

  Wire.begin(D0, D1);

  pwm.begin();
  pwm.setPWMFreq(50);

  if (!lox.begin()) {
    Serial.println("ToF init failed");
    robotState = ERROR_STATE;
  }

  if (!ina219.begin()) {
    Serial.println("INA219 init failed");
  }

  tft.init();
  tft.setRotation(1);

  setServoAngle(PAN_CH, 90);
  setServoAngle(TILT_CH, 90);
  armsHome();

  // Add Wi-Fi / WebServer initialization here.
}

void loop() {
  updateSensors();
  processCommands();
  safetyCheck();
  serviceWebServer();
}
```

Key rule

There is exactly one setup() and one loop(). Feature code becomes functions called from them; do not paste separate setup() blocks from individual sections into the same sketch.

REVISION 2 — 8. Firmware: Motors + Arms

Motor driver

```
void driveMotor(bool left, int8_t direction, uint16_t speed) {
  uint8_t inA = left ? L_AIN1 : R_BIN1;
  uint8_t inB = left ? L_AIN2 : R_BIN2;

  speed = constrain(speed, 0, 4095);

  if (direction > 0) {
    pwm.setPWM(inA, 0, speed);
    pwm.setPWM(inB, 0, 0);
  } else if (direction < 0) {
    pwm.setPWM(inA, 0, 0);
    pwm.setPWM(inB, 0, speed);
  } else {
    pwm.setPWM(inA, 0, 0);
    pwm.setPWM(inB, 0, 0);
  }
}

void stopMotors() {
  driveMotor(true, 0, 0);
  driveMotor(false, 0, 0);
}
```

Arm animations

```
void performWaveAnimation() {
  robotState = WAVING;

  // Right arm performs the wave; left arm stays neutral.
  setArmAngleSafe(RIGHT_ARM_CH, 120);
  delay(250);

  for (int i = 0; i < 3; i++) {
    setArmAngleSafe(RIGHT_ARM_CH, 75);
    delay(180);
    setArmAngleSafe(RIGHT_ARM_CH, 120);
    delay(180);
  }

  setArmAngleSafe(RIGHT_ARM_CH, ARM_HOME);
  robotState = IDLE;
}

void performArmsUpAnimation() {
  robotState = ARMS_UP;

  setArmAngleSafe(LEFT_ARM_CH, ARM_UP);
  setArmAngleSafe(RIGHT_ARM_CH, ARM_UP);
  delay(500);

  armsHome();
  robotState = IDLE;
}

void armsDown() {
  setArmAngleSafe(LEFT_ARM_CH, ARM_DOWN);
  setArmAngleSafe(RIGHT_ARM_CH, ARM_DOWN);
}
```

Calibration note: if your physical horn orientation is mirrored, do not change the safety limits first. Reverse the mapping for that servo or adjust the horn installation, then re-check the 45°–135° mechanical envelope.

REVISION 2 — 8. Firmware: Sensors + Safety

Battery monitor

```
float readBatteryVoltage() {
  // INA219 must be installed on the battery-side measurement path.
  return ina219.getBusVoltage_V();
}

float readBatteryPercent() {
  float v = readBatteryVoltage();

  // Simple Li-ion approximation; calibrate for the chosen cell/load.
  float pct = (v - 3.20f) / (4.20f - 3.20f) * 100.0f;
  return constrain(pct, 0.0f, 100.0f);
}
```

Obstacle safety

```
const uint16_t SAFE_DISTANCE_MM = 200;

bool obstacleTooClose() {
  VL53L0X_RangingMeasurementData_t m;
  lox.rangingTest(&m, false);

  if (m.RangeStatus == 4) return false; // invalid reading
  return m.RangeMilliMeter < SAFE_DISTANCE_MM;
}

void safetyCheck() {
  if (robotState == MOVING && obstacleTooClose()) {
    stopMotors();
    robotState = AVOIDING;
  }

  float battery = readBatteryPercent();

  if (battery < 10.0f) {
    stopMotors();
    robotState = ERROR_STATE;
  }
}
```

Command layer — AI does not touch hardware directly

```
enum CommandType {
  CMD_NONE,
  CMD_FORWARD,
  CMD_BACKWARD,
  CMD_LEFT,
  CMD_RIGHT,
  CMD_STOP,
  CMD_WAVE,
  CMD_ARMS_UP,
  CMD_ARMS_HOME,
  CMD_LOOK_LEFT,
  CMD_LOOK_RIGHT
};

void executeCommand(CommandType cmd) {
  switch (cmd) {
    case CMD_FORWARD:
      robotState = MOVING;
      driveMotor(true,  1, 2200);
      driveMotor(false, 1, 2200);
      break;

    case CMD_STOP:
      emergencyStop();
      break;

    case CMD_WAVE:
      performWaveAnimation();
      break;

    case CMD_ARMS_UP:
      performArmsUpAnimation();
      break;

    case CMD_ARMS_HOME:
      armsHome();
      break;

    default:
      break;
  }
}
```

The same command functions are used by the web dashboard and by the AI intent layer. This keeps behavior consistent and makes safety checks centralized.

REVISION 2 — 8. Firmware: Web Dashboard

```
#include <WebServer.h>
WebServer server(80);

void handleWave() {
  performWaveAnimation();
  server.send(200, "text/plain", "Waved");
}

void handleArmsUp() {
  performArmsUpAnimation();
  server.send(200, "text/plain", "Arms up");
}

void handleArmsHome() {
  armsHome();
  server.send(200, "text/plain", "Arms home");
}

void handleStop() {
  emergencyStop();
  server.send(200, "text/plain", "Stopped");
}

void setupWebServer() {
  server.on("/wave", handleWave);
  server.on("/armsup", handleArmsUp);
  server.on("/armshome", handleArmsHome);
  server.on("/stop", handleStop);
  server.begin();
}

void serviceWebServer() {
  server.handleClient();
}
```

Dashboard controls

Button	Endpoint	Function
Wave (Arms)	/wave	performWaveAnimation()
Arms Up	/armsup	performArmsUpAnimation()
Arms Home	/armshome	armsHome()
STOP	/stop	emergencyStop()

Non-blocking upgrade for final firmware

The examples above use short delays to make the behavior easy to understand. For the final integrated robot, replace long delay()-based animations with a timed state machine so the web server, ToF safety, battery monitoring, and audio remain responsive while an arm is moving.

AI / voice integration boundary

Keep the OpenAI transport layer separate from hardware control. The AI returns a validated intent such as WAVE, MOVE_FORWARD, LOOK_LEFT, or STOP; the local firmware decides whether and how to execute it. Never hardcode a secret API key into a distributable firmware image.

9. 3D Printing / STL Guidance

Fi 4 STL blanks jahzin (men chat sabe2, mershabin bel-2ab3ad el-muṣaḥḥaḥa): `head_box.stl` (60×35×45mm), `neck_platform.stl` (29×20×3mm), `trapezoidal_body.stl` (qa3ide 63mm → foo2 29mm), `base_tracks.stl` (63×65×32mm).

> **Na2es:** arm mount brackets w el-arm segments zetohon mesh mawjoudin bel-blanks el-jahzin. Lezim tzidon yadawiyyan (Seksyon 4.1 fiya el-spec: 2 fatḥat servo horn 3al janbein el-trapezoidal_body.stl, w dhira3 brass wire/PLA taqriban 3-3.5cm).

> Hayde solid boxes be-l-2ab3ad el-khareji el-mazbouṭa bas — el-slicer (Cura/PrusaSlicer) howe yalli byedir el-shell thickness w el-infill. El-cutouts (shashet TFT, 3adaset camera, thoqoub speaker, fatḥet servo) lazim tzidon b-Tinkercad/Fusion360/Blender aw b-slicer's cut tool.

Khatawat el-Ta3dil (Tinkercad — sahle lal-mubtedi2in)

1. tinkercad.com → hesab majjani → "Create new design"
2. Ḍbeṭ waḥdet el-qiyas 3ala centimeter
3. Este3mel "Shell" tool la-tfarrigh kel qeṭ3a (byewaffer madde w yekhaffef el-wazn)
4. Ḍif fat-ḥat/dawa2er lal-cutouts (shashe, camera, speaker, servo horn)
5. Export → STL, w eftaḥ b-Cura/PrusaSlicer: Infill 20-25% Gyroid, Layer height 0.12-0.16mm

Badil asra3: dawwer 3ala Thingiverse/Printables 3an "small WALL-E robot head pan tilt" aw "mini robot chassis servo" w 3addel el-qiyasat 3ala el-namouzej el-jahez badal ma tebda2 men el-ṣifr.

REVISION 2 — 8. Firmware Integration Checklist

Master .ino integration order

Order	Module	Pass condition
1	I²C scan	0x40 PCA9685, 0x41 INA219, 0x29 ToF detected
2	PCA9685	PWM frequency = 50 Hz; channels respond
3	Arms	CH6/CH7 reach HOME safely without binding
4	Head	CH0/CH1 center at 90° and move within mechanical limits
5	Motors	DRV8833 responds to forward/reverse/stop
6	ToF	Obstacle < SAFE_DISTANCE triggers motor stop
7	INA219	Battery-side voltage/current reading plausible
8	Display	ST7789 initializes with correct pins/driver
9	Audio	MAX98357A output works without brownout
10	Dashboard	Wave / Arms Up / Arms Home / STOP work
11	AI	Voice intent calls command layer only

Final architecture

```
VOICE / WEB / LOCAL INPUT
          |
          v
     COMMAND / INTENT
          |
          v
   SAFETY + STATE MACHINE
          |
    +-----+------+------+
    |            |      |
  MOTORS       HEAD    ARMS
  CH2-CH5      CH0-1   CH6-7
    |            |      |
 DRV8833      PCA9685  PCA9685
    |
  N20 TRACKS

Sensors -> updateSensors() -> state/safety
Battery -> INA219 -> safety/status
ToF -> obstacle check -> motor stop
```

Build rule: freeze the mechanical dimensions and pinout before printing final parts. Then validate one subsystem at a time. The first complete milestone is: Move + Look + Wave + Speak/Hear + Detect obstacle + Stop.

REVISION 2 — 10. Final Checklist

Mechanical

☐ Head / neck / body / track dimensions confirmed in CAD
☐ Neck width fixed after servo clearance check
☐ Arm brackets + arm links are separate CAD/STL parts
☐ SG90 horn centered at 90° before installation
☐ 45°–135° arm sweep verified manually
☐ ≥5 mm clearance from tracks/body/screen
☐ Arm moving mass kept light

Electrical

☐ MT3608 adjusted to 5.0V before connection
☐ Common GND verified
☐ INA219 installed on the battery-side measurement path
☐ PCA9685 = 0x40 / INA219 = 0x41 / ToF = 0x29
☐ 100–470 µF bulk capacitor near servo rail
☐ DRV8833 motor rail kept separate from logic/servo current path

Firmware

☐ One master .ino with one setup() and one loop()
☐ CH6/CH7 arm functions integrated
☐ Servo limits centralized at 45°–135°
☐ Motor speed is parameterized
☐ Emergency stop implemented
☐ ToF obstacle stop implemented
☐ Battery low-voltage behavior implemented
☐ Dashboard Wave / Arms Up / Arms Home / STOP tested
☐ AI calls the command layer rather than raw PWM

Release milestone

Do not add more features until the base robot passes: Move → Look → Wave → Speak/Hear → Detect obstacle → Stop.

This REV2 preserves the original component choices and dimensions while making the arm geometry, power routing, firmware integration, safety limits, and dashboard control explicit.